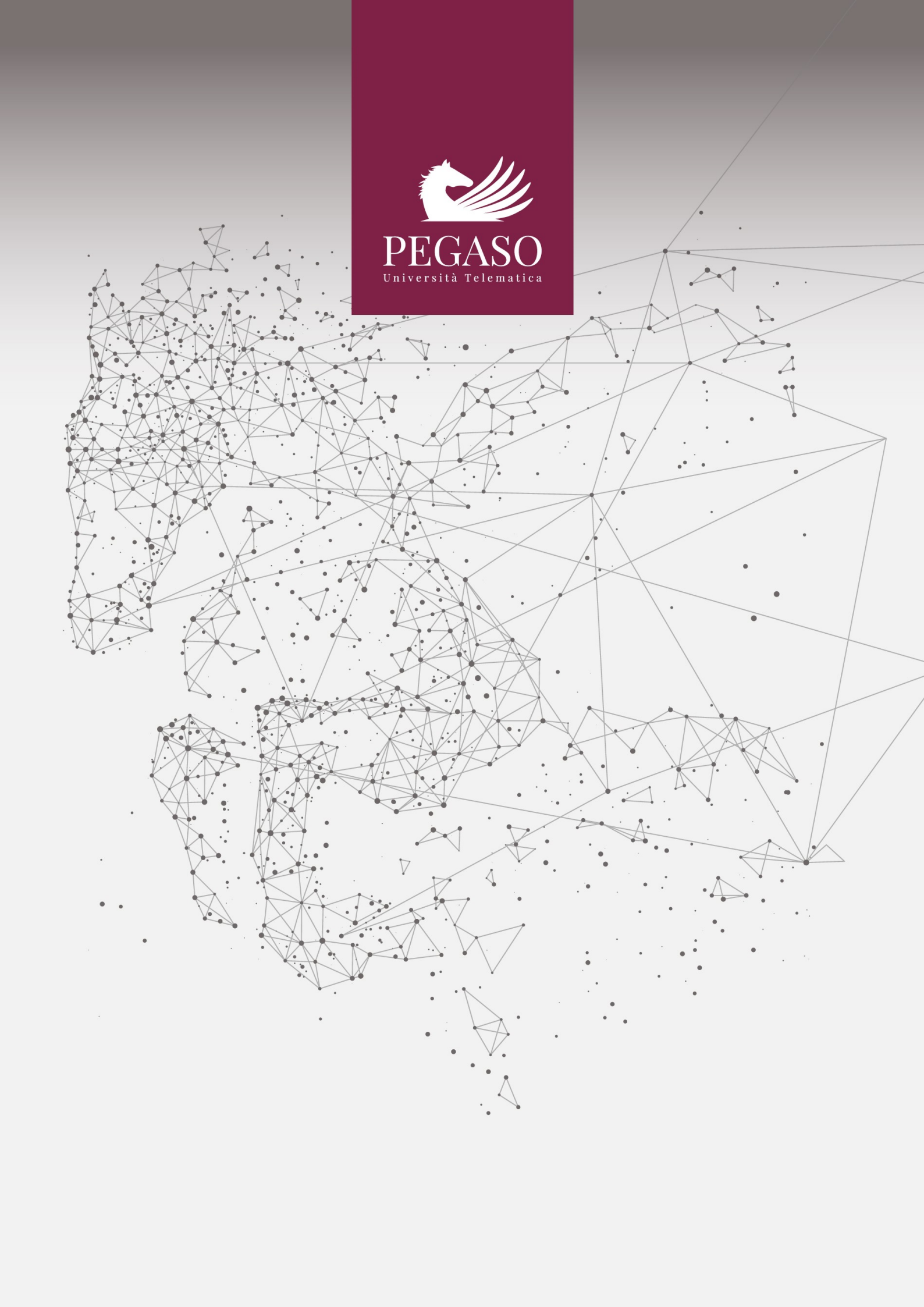

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. VERIFICA DATA....4
3. SOMMA E SOTTRAZIONE DI GIORNI A DATE....11
4. CALCOLATRICE PER LE DATE....17
BIBLIOGRAFIA....20

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano ed analizzano alcune esercitazioni molto importanti che approfondiscono il concetto di lista nei calcoli relativi al tempo in termini di date. In particolare, si riportano quattro esercizi in cui si risolvono tipici problemi di calcoli temporali. Il primo esercizio mostra l'utilità delle liste per i controlli sulle date. Il secondo esercizio mostra come calcolare il numero dei giorni mancanti al termine dell'anno a partire da una data. Il terzo esercizio illustra le funzioni definite dal programmatore per risolvere il problema della somma e della sottrazione di giorni alle date. Il quarto e ultimo esercizio mostra come mettere a disposizione dell'utente tutte le funzioni scritte negli esercizi precedenti per ottenere una calcolatrice di date che offre cinque funzionalità diverse.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Verifica data

L'esercizio che si propone è quello di verificare una data compresa tra il 1° gennaio del 1900 e il 31 dicembre del 2030 considerando anche gli anni bisestili. I controlli da effettuare sono diversi:

1) di base il giorno deve essere compreso tra 1 e 31, il mese tra 1 e 12 e l'anno (per questo esercizio in particolare) tra 1900 e 2030 estremi inclusi. Questo controllo può essere fatto direttamente nel *main()* in fase di input dei dati attraverso cicli di validazione dell'input (un ciclo per il giorno, uno per il mese, uno per l'anno);
2) il giorno dovrà essere minore o uguale all'ultimo giorno del mese della data (es. ad Aprile il giorno deve essere minore o uguale a 30 e non a 31). Parte di questo controllo può essere effettuata grazie alla funzione *g_mesi()* che restituisce il numero dei giorni di un mese. Rispetto alla normale funzione *g_mesi()*, però, per il controllo si imporrà Febbraio a 29 giorni in modo tale da validare comunque il giorno 29 per il mese di Febbraio, lasciando ad un ulteriore controllo la verifica sugli anni bisestili. Nel codice Python proposto, che segue, la funzione che realizza l'intera verifica in questione è *errore_gm()* che prende come parametri il giorno e il mese della data inserita;
3) per i soli anni non bisestili e per le date del mese di Febbraio, si deve verificare che il giorno non sia il 29 poiché errato. Parte di questo controllo può essere effettuata grazie alla funzione *bisestile()* che verifica se un particolare anno sia bisestile o meno. Nel codice Python proposto, che segue, la funzione che realizza la verifica in questione è *errore_data()* che prende come parametri l'intera data inserita (giorno, mese, anno).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```python
def g_mesi(n):
    giorni_mesi = [31,29,31,30,31,30,31,31,30,31,30,31]
    return giorni_mesi[n-1]

def bisestile(anno):
    return (anno%4 == 0) and (anno%100 != 0) or (anno%400 == 0)

def errore_gm(g,m):
    if g<=g_mesi(m):
        return False  # non c'è errore
    else:
        return True   # c'è errore

def errore_data(g,m,a):
    if not(bisestile(a)) and m==2:
        if g==29:
            return True   #c'è errore - anno non bisestile
    else:
        return False  # non c'è errore
```

Nel *main()* si inizializzano giorno, mese e anno a numeri errati per poter entrare nel ciclo di validazione dell'input di giorno, mese e anno. Si utilizza, inoltre, un flag (inizializzato a False) per impostare una struttura iterativa che termina solo quando la data inserita supera tutti i controlli ed è validata.

Oltre al controllo di base sull'intervallo per giorno, mese e anno realizzato tramite cicli di validazione dell'input, l'effettiva validazione della data avviene solo se le funzioni di errore (*errore_gm()* e *errore_data()*) restituiscono entrambe False.

Si ricorda che l'operatore logico or tra due variabili booleane restituisce True se anche solo una delle due variabili è True. Segue, quindi, che basta un controllo non passato porta a non validare la data.

Di seguito si riporta il codice Python della funzione *main()* che richiama le funzioni analizzate in precedenza e implementa l'intero algoritmo di validazione della data.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```python
def main():
    print("VERIFICA DATA")

    g=40
    m=40
    a=-10
    data_ok = False
    print("Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030")
    while data_ok == False:
        while g<=0 or g>31:
            g = int(input("Inserire giorno: "))
        while m<=0 or m>12:
            m = int(input("Inserire mese: "))
        while a<1990 or a>2030:
            a = int(input("Inserire anno: "))
        if errore_gm(g,m) or errore_data(g,m,a):
            print("Data inserita non compresa nell'intervallo")
            g=40
            m=40
            a=-10
        else:
            data_ok = True
            print("Data valida")


main()
```

Di seguito si riporta il risultato dell'esecuzione del codice Python per quattro date: 15/10/1999 (per cui in fase di digitazione sono stati provocati appositamente degli errori per mostrare il funzionamento dei cicli di validazione dell'input), 31/04/2000 (data non esistente poiché Aprile è formato da 30 giorni), 29/02/2023 (data non esistente perché il 2023 non è un anno bisestile), 29/02/2024 (data valida poiché il 2024 è un anno bisestile).

```
VERIFICA DATA
Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2023
Inserire giorno: 35
Inserire giorno: 15
Inserire mese: 20
Inserire mese: 22
Inserire mese: 10
Inserire anno: 1999
Data valida
>>>
VERIFICA DATA
Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030
Inserire giorno: 31
Inserire mese: 04
Inserire anno: 2000
Data inserita non compresa nell'intervallo
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
Inserire giorno: 29
Inserire mese: 02
Inserire anno: 2023
Data inserita non compresa nell'intervallo
Inserire giorno: 29
Inserire mese: 02
Inserire anno: 2024
Data valida
```

- All'interno di una funzione per poter permettere l'input della data stessa;
- dovrà contenere una funzione che, a partire dalla data valida inserita, restituirà il numero dei giorni mancanti fino a fine anno considerando anche gli anni bisestili.

Il main() del programma è organizzato per essere semplice e lineare, demandando la complessità dell'esercizio a due funzioni:

- insert_data() che restituisce una data corretta e valida;
- end_data() che accetta come parametri giorno, mese e anno della data inserita e restituisce i giorni mancanti al termine dell'anno.

Segue il codice Python della funzione main() e delle funzioni necessarie per l'inserimento e la validazione della data.

```python
def main():
    print("GIORNI MANCANTI")
    g,m,a = insert_data()
    print("Il numero dei giorni che mancano al 31/12/",a," è pari a ",end_data(g,m,a)," giorni",sep="")

main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def g_mesi(n):
    giorni_mesi = [31,28,31,30,31,30,31,31,30,31,30,31]
    return giorni_mesi[n-1]


def bisestile(anno):
    return (anno%4 == 0) and (anno%100 != 0) or (anno%400 == 0)


def errore_gm(g,m):
    if g<=g_mesi(m):
        return False  # non c'è errore
    else:
        return True   # c'è errore


def errore_data(g,m,a):
    if not(bisestile(a)) and m==2:
        if g==29:
            return True   #c'è errore - anno non bisestile
    else:
        return False  # non c'è errore
```

```
def insert_data():
    g=40
    m=40
    a=-10
    data_ok = False
    print("Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030")
    while data_ok == False:
        while g<=0 or g>31:
            g = int(input("Inserire giorno: "))
        while m<=0 or m>12:
            m = int(input("Inserire mese: "))
        while a<1990 or a>2030:
            a = int(input("Inserire anno: "))
        if not(errore_gm(g,m) or errore_data(g,m,a)):
            return g,m,a
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione end_data() che prende come parametri giorno g, mese m e anno a della data da considerate opera nel seguente modo:

- se il mese della data in questione è Dicembre, la funzione restituirà i giorni che mancano al 31/12, quindi 31-g;
- per gli altri mesi (es. Aprile):
  - si aggiungono i giorni dei mesi successivi al mese della data inserita (es. da Maggio a Dicembre);
  - si aggiungono i giorni che mancano alla fine del mese (es. giorni che mancano al 30/04), quindi g_mesi(m)-g;
  - se il mese della data in questione è Gennaio o Febbraio, si considera se l'anno è bisestile e, nel caso, si aggiunge un giorno nel calcolo dei giorni mancanti.

Segue il codice Python della funzione end_data().

```
def end_data(g,m,a):
    if m==12:
        return g_mesi(12)-g
    else:
        mancanti = 0
        for i in range(12,m,-1):
            mancanti+=g_mesi(i)
        mancanti+=g_mesi(m)-g
        if bisestile(a) and m<=2:
            mancanti+=1
        return mancanti
```

Di seguito si riporta il risultato dell'esecuzione del codice Python per la data di esempio 22/03/2023.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
GIORNI MANCANTI
Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2023
Inserire giorno: 22
Inserire mese: 03
Inserire anno: 2023
Il numero dei giorni che mancano al 31/12/2023 è pari a 284 giorni
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Somma e sottrazione di giorni a date

Gli esercizi che si propongono sono relativi alla somma e alla sottrazione di giorni ad una data. In particolare, verrà trattato prima il caso di somma e poi quello di sottrazione di giorni.

Ad ogni modo, in entrambi i casi si utilizzeranno funzioni già esaminate:

- g_mesi() per sapere il numero dei giorni di un mese. in questo caso si considera il numero dei giorni di Febbraio pari a 28. Nel caso di anni bisestili, si provvede con una correzione del numero di giorni da considerare;
- bisestile() per sapere se l'anno considerato è bisestile o no;
- errore_gm(), errore_data() e insert_data() per permettere l'inserimento di date valide e controllate.

Per maggiore chiarezza, segue il codice Python delle funzioni appena elencate.

```
def g_mesi(n):
    giorni_mesi = [31,28,31,30,31,30,31,31,30,31,30,31]
    return giorni_mesi[n-1]


def bisestile(anno):
    return (anno%4 == 0) and (anno%100 != 0) or (anno%400 == 0)


def errore_gm(g,m):
    if g<=g_mesi(m):
        return False  # non c'è errore
    else:
        return True   # c'è errore


def errore_data(g,m,a):
    if not(bisestile(a)) and m==2:
        if g==29:
            return True   #c'è errore - anno non bisestile
    else:
        return False  # non c'è errore
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def insert_data():
    g=40
    m=40
    a=-10
    data_ok = False
    print("Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030")
    while data_ok == False:
        while g<=0 or g>31:
            g = int(input("Inserire giorno: "))
        while m<=0 or m>12:
            m = int(input("Inserire mese: "))
        while a<1990 or a>2030:
            a = int(input("Inserire anno: "))
        if not(errore_gm(g,m) or errore_data(g,m,a)):
            return g,m,a
```

Il main() del programma è organizzato per essere semplice e lineare, demandando la complessità dell'esercizio a due funzioni:

- insert_data() che restituisce una data corretta e valida;
- somma_data() che accetta come parametri giorno, mese e anno della data inserita e il numero dei giorni da sommare e restituisce giorno, mese e anno della nuova data.

  Segue il codice Python della sola funzione main().

```
def main():
    print("SOMMA DI GIORNI AD UNA DATA")
    g,m,a = insert_data()
    dd = int(input("Inserire numero giorni da sommare alla data: "))
    g1,m1,a1 = somma_data(g,m,a,dd)
    print("La nuova data è ",g1,"/",m1,"/",a1,sep="")


main()
```

Prima di progettare la funzione somma_data(), è molto utile codificare la funzione increm_mese() che accetta come parametri mese e anno della data inserita e restituisce mese e anno opportunamente incrementati di un mese. Tale funzione incrementa di 1 il mese passato come parametro, tenendo conto del caso in cui m sia l'ultimo mese dell'anno. In questo caso, la funzione provvede a riportare il mese a 1 e ad incrementare l'anno a di 1 come è possibile osservare dal codice seguente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def decrem_mese(m,a):
    if m==1:
        a-=1
        m=12
    else:
        m-=1
    return m,a
```

La funzione somma_data() che accetta come parametri giorno g, mese m e anno a della data inserita e il numero dei giorni da sommare dd, opera nel seguente modo:

- calcola il numero dei giorni diff che mancano al termine del mese m correggendo tale numero nel caso di m=2 e di anno bisestile;
- se il numero dei giorni da sommare dd è inferiore al numero dei giorni diff che mancano al termine del mese, la funzione restituirà come giorno della nuova data g+dd, mentre come mese m e come anno a;
- in caso contrario, giorni da sommare dd maggiori del numero dei giorni diff che mancano alla fine del mese (dd>diff):
    - si decrementa dd della quantità pari a diff per arrivare al termine del mese m;
    - si incrementa il mese mediante la funzione increm_mese();
    - si porta il giorno a 1;
    - si «consumano» i giorni da aggiungere dd sottraendo i giorni dei mesi che seguono e incrementando il mese (e di conseguenza anche l'anno ogni Dicembre), finché dd non è minore o uguale a 31;
    - nel caso in cui il numero degli ultimi giorni da aggiungere contenuti in dd fosse effettivamente maggiore del numero dei giorni del mese, sarebbe necessario incrementare di 1 il mese con la funzione increm_mese();
    - mentre mese m e anno a sono aggiornati alla nuova data, il giorno g della nuova data si pone uguale a dd.

Segue il codice Python della funzione somma_data() che implementa l'algoritmo appena analizzato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def decrem_data(g,m,a,dd):
    if dd<g:
        return g-dd,m,a
    else:                   #si passa all'ultimo giorno del mese precedente
        dd-=g               #se il mese è gennaio scatta anno
        m,a = decrem_mese(m,a)
        #g=g_mesi(m)
        #print("GIORNO:",g,"\tMESE:",m,"\tANNO:",a,"\tDD=",dd)

        while dd>30:            #per "consumare" mesi con avanzamento anni finchè non si raggiunge un numero di giorni
            dd-=g_mesi(m)       #inferiore a 31
            if m==2 and bisestile(a):
                dd-=1
            m,a = decrem_mese(m,a)
            #g=g_mesi(m)
            #print("GIORNO:",g,"\tMESE:",m,"\tANNO:",a,"\tDD=",dd)

        if dd>g_mesi(m):
            dd-=g_mesi(m)
            m,a = decrem_mese(m,a)

        g=g_mesi(m)-dd
        return g,m,a
```

Di seguito si riporta il risultato dell'esecuzione del codice Python completo per la data di esempio 15/01/2001 e un numero di giorni da sommare pari a 730 (equivalenti a 2 anni).

```
SOMMA DI GIORNI AD UNA DATA
Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030
Inserire giorno: 15
Inserire mese: 01
Inserire anno 2001
Inserire numero giorni da sommare alla data: 730
La nuova data è 15/1/2003
```

Passando all'algoritmo di sottrazione di giorni ad una data, il main() del programma sarà nuovamente organizzato per essere semplice e lineare, demandando la complessità dell'esercizio a due funzioni:

- insert_data() che restituisce una data corretta e valida;
- decrem_data() che accetta come parametri giorno, mese e anno della data inserita e il numero dei giorni da sottrarre e restituisce giorno, mese e anno della nuova data.

  Segue il codice Python della sola funzione main().

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    print("SOTTRARRE GIORNI AD UNA DATA")
    g,m,a = insert_data()
    dd = int(input("Inserire numero giorni da sottrarre alla data: "))
    g1,m1,a1 = decrem_data(g,m,a,dd)
    print("La nuova data è ",g1,"/",m1,"/",a1,sep="")


main()
```

Prima di progettare la funzione decrem_data(), è molto utile codificare la funzione decrem_mese() che accetta come parametri mese e anno della data inserita e restituisce mese e anno opportunamente decrementati di un mese. Tale funzione decrementa di 1 il mese passato come parametro, tenendo conto del caso in cui m sia il primo mese dell'anno. In questo caso, provvede a riportare il mese a 12 ed a decrementare l'anno a di 1 come è possibile osservare dal codice seguente.

```
def decrem_mese(m,a):
    if m==1:
        a-=1
        m=12
    else:
        m-=1
    return m,a
```

La funzione decrem_data() che accetta come parametri giorno g, mese m e anno a della data inserita e il numero dei giorni da sottrarre dd, opera nel seguente modo:

- se il numero dei giorni da decrementare è inferiore al giorno g della data, restituisce lo stesso mese m ed anno a, con il giorno pari a g-dd;
- in caso contrario, numero dei giorni da decrementare dd maggiore o uguale al numero del giorno (dd>=g):
    - si decrementa dd della quantità pari a g per arrivare al termine del mese precedente;
    - si decrementa il mese mediante la funzione decrem_mese();
    - si «consumano» i giorni da sottrarre dd sottraendo i giorni dei mesi che precedono e decrementando il mese (e di conseguenza anche l'anno ogni Gennaio), finché dd non è minore di 30;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- nel caso in cui il numero degli ultimi giorni da aggiungere contenuti in dd fosse effettivamente maggiore del numero dei giorni del mese, sarebbe necessario decrementare di 1 il mese con la funzione decrem_mese();
- mentre mese m e anno a sono aggiornati alla nuova data, il giorno g della nuova data si pone uguale alla differenza tra il numero dei giorni del mese m e il numero residuo di dd.

Segue il codice Python della funzione decrem_data() che implementa l'algoritmo appena analizzato.

```
def decrem_data(g,m,a,dd):
    if dd<g:
        return g-dd,m,a
    else:                   #si passa all'ultimo giorno del mese precedente
        dd-=g               #se il mese è gennaio scatta anno
        m,a = decrem_mese(m,a)
        #g=g_mesi(m)
        #print("GIORNO:",g,"\tMESE:",m,"\tANNO:",a,"\tDD=",dd)

        while dd>30:            #per "consumare" mesi con avanzamento anni finchè non si raggiunge un numero di giorni
            dd-=g_mesi(m)       #inferiore a 31
            if m==2 and bisestile(a):
                dd-=1
            m,a = decrem_mese(m,a)
            #g=g_mesi(m)
            #print("GIORNO:",g,"\tMESE:",m,"\tANNO:",a,"\tDD=",dd)

        if dd>g_mesi(m):
            dd-=g_mesi(m)
            m,a = decrem_mese(m,a)

        g=g_mesi(m)-dd
        return g,m,a
```

Di seguito si riporta il risultato dell'esecuzione del codice Python completo per la data di esempio 20/03/2023 e un numero di giorni da sottrarre pari a 167.

```
SOTTRARRE GIORNI AD UNA DATA
Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030
Inserire giorno: 20
Inserire mese: 03
Inserire anno: 2023
Inserire numero giorni da sottrarre alla data: 167
La nuova data è 4/10/2022
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Calcolatrice per le date

Al fine di riunire tutte le funzionalità relative ai calcoli di tempo sviluppate finora, si propone l'esercizio della calcolatrice per le date. In particolare, il programma conterrà tutte le funzioni sviluppate e metterà a disposizione delle funzionalità relative ai calcoli temporali.

Le operazioni disponibili sono:

- numero dei giorni passati dall'inizio dell'anno;
- numero dei giorni che mancano alla fine dell'anno;
- distanza tra due date;
- sommare giorni ad una data;
- sottrarre giorni ad una data.

Attraverso la selezione numerica della funzionalità, sarà possibile procedere all'input, all'elaborazione e all'output relativi

Le funzioni che verranno impiegate in questo esercizio sono:

- g_mesi() che restituisce il numero dei giorni del mese passato come parametro;
- bisestile() che indica se l'anno passato come parametro è bisestile o meno;
- errore_gm(), errore_data() ed insert_data() che permettono di inserire una data valida;
- ord_data() che permette di sapere quanti giorni sono passati dall'inizio dell'anno a partire da una data espressa come giorno, mese e anno;
- end_data() che permette di sapere quanti giorni mancano alla fine dell'anno a partire da una data espressa come giorno, mese e anno;
- calcola_diff_g_mese(), calcola_diff_g_anno() e diff_data() che permettono di calcolare il numero dei giorni che intercorrono tra due date;
- increm_mese() e somma_data() che permettono di sommare un certo numero di giorni ad una data espressa come giorno, mese e anno;
- decrem_mese() e decrem_data() che permettono di sottrarre un certo numero di giorni ad una data espressa come giorno, mese e anno.

Il main() del programma, attraverso una catena di if-elif inserita in una struttura iterativa, permette la selezione di una delle cinque funzionalità previste dalla calcolatrice più la possibilità di uscire dal programma. Di seguito si riporta l'aspetto del menu stampato dalla funzione main per permettere la scelta della funzionalità di calcolo delle date.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
CALCOLATRICE DI DATE
```

1. Numero dei giorni passati dall'inizio dell'anno
2. Numero dei giorni che mancano alla fine dell'anno
3. Distanza tra due date
4. Sommare giorni ad una data
5. Sottrarre giorni ad una data
6. Uscire

Segue il codice Python della funzione main() che implementa il menu e richiama opportunamente le funzioni riguardanti il tempo analizzate finora.

```
def main():
    print("\n\t\tCALCOLATRICE DI DATE\n\n")
    i=0
    while i!=6:
        print("\t\tMENU")
        print("1. Numero dei giorni passati dall'inizio dell'anno")
        print("2. Numero dei giorni che mancano alla fine dell'anno")
        print("3. Distanza tra due date")
        print("4. Sommare giorni ad una data")
        print("5. Sottrarre giorni ad una data")
        print("6. Uscire")
        i= int(input("\nInserire la scelta: "))
```

```
    if i==1:
        g,m,a = insert_data()
        print("Il numero dei giorni passati dal 01/01/",a," è pari a ",ord_data(g,m,a)," giorni",sep="")
    elif i==2:
        g,m,a = insert_data()
        print("Il numero dei giorni che mancano al 31/12/",a," è pari a ",end_data(g,m,a)," giorni",sep="")
    elif i==3:
        print("PRIMA DATA")
        g,m,a = insert_data()
        print("SECONDA DATA")
        g1,m1,a1 = insert_data()
        print("La distanza tra il",g,"/",m,"/",a," e il ",g1,"/",m1,"/",a1," è pari a ",diff_data(g,m,a,g1,m1,a1)," giorni",sep="")
    elif i==4:
        g,m,a = insert_data()
        dd = int(input("Inserire numero giorni da sommare alla data: "))
        g1,m1,a1 = somma_data(g,m,a,dd)
        print("La nuova data è ",g1,"/",m1,"/",a1,sep="")
    elif i==5:
        g,m,a = insert_data()
        dd = int(input("Inserire numero giorni da sottrarre alla data: "))
        g1,m1,a1 = decrem_data(g,m,a,dd)
        print("La nuova data è ",g1,"/",m1,"/",a1,sep="")
    elif i==6:
        print("Esco dal programma...")
    else:
        print("Scelta non valida")
    print("")

main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito si riporta il risultato dell'esecuzione del codice Python per l'esecuzione della funzionalità n. 3 (calcolo della distanza tra due date) e successiva scelta di uscire dal programma.

```
CALCOLATRICE DI DATE
   1. Numero dei giorni passati dall'inizio dell'anno
   2. Numero dei giorni che mancano alla fine dell'anno
   3. Distanza tra due date
   4. Sommare giorni ad una data
   5. Sottrarre giorni ad una data
   6. Uscire
Inserire la scelta: 3
PRIMA DATA
Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030
Inserire giorno: 01
Inserire mese: 01
Inserire anno: 1995
SECONDA DATA
Inserire una data compresa nell'intervallo 01/01/1990 e 31/12/2030
Inserire giorni: 22
Inserire mese: 05
Inserire anno: 2015
La distanza tra il 1/1/1995 e il 22/5/2015 è pari a 7445 giorni
MENU
   1. Numero dei giorni passati dall'inizio dell'anno
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).